

Мастер-план тестирования (Master Test Plan)

Проект: TIMP-project-server
Основан на стандарте ISO/IEC/IEEE 29119-3

Ответственный за план: «Садыков Ростислав Андреевич»

24 апреля 2026 г.

Содержание

1	Введение	2
2	Объекты тестирования	2
3	Функции, подлежащие тестированию	2
4	Функции, НЕ подлежащие тестированию	2
5	Подход к тестированию	3
6	Критерии прохождения	3
7	Фаззинг-тестирование и безопасность	3
7.1	Задачи разработчика по фаззингу	4
8	Требования к тестовой среде	4
9	Обязанности	4
10	Расписание и Риски	4

1 Введение

Данный документ представляет собой формализованный план тестирования серверной части проекта **TIMP-project-server**. Сервер реализован на C++ с использованием фреймворка Qt6 (сетевое взаимодействие через QTcpServer, парсинг JSON), взаимодействует с СУБД PostgreSQL и внешним API Brevo.

Цель тестирования — гарантировать корректность вычислений, отказоустойчивость сетевого слоя при фрагментации TCP и защиту от атак. Если код не покрыт тестами, он считается **нерабочим**.

2 Объекты тестирования

- **Модуль математики:** `src/calc.cpp`
- **Модуль авторизации:** `src/AuthManager.cpp`
- **Сетевой слой и маршрутизация:** `src/Server.cpp`

3 Функции, подлежащие тестированию

1. **Бизнес-логика:** Математические расчеты с учетом краевых значений ($x < 0$, $x = 0$, $0 < x < 1$, $x = 1$, $x > 1$) и защиты от деления на ноль.
2. **Логика БД:** Успешное сохранение и извлечение учетных данных пользователей, корректная работа с транзакциями.
3. **Безопасность:** Хеширование паролей, ограничение времени жизни кодов верификации (TTL), защита от брутфорса, валидация длины и сложности паролей.
4. **Сетевая стабильность:** Обработка разорванных пакетов TCP, некорректного JSON, переполнения буфера.

4 Функции, НЕ подлежащие тестированию

- Внутренняя логика стороннего API (Brevo). Для тестирования отправки email будут использоваться заглушки.
- Стабильность самого ядра СУБД PostgreSQL (тестируется только корректность SQL-запросов сервера).

5 Подход к тестированию

Общая стратегия строится на пирамиде тестирования:

- **Unit-тестирование:** Используется встроенный модуль **Qt Test**. Разработчики обязаны написать тесты на каждую публичную функцию классов.
- **Интеграционное тестирование:** Автоматические скрипты (например, на Python + pytest), поднимающие тестовую БД в Docker и дергающие TCP-сокеты реального сервера.
- **Фаззинг:** См. Раздел 7. Обязательный этап для проверки обработки JSON и сетевых потоков.

6 Критерии прохождения

- **Pass:** 100% Unit-тестов выполняются успешно. Покрытие кода (измеряется через gcov/lcov) составляет не менее 85%. Фаззер работает 24 часа без единого падения сервера или утечки памяти (проверяется через Valgrind).
- **Fail:** Наличие хотя бы одного крэша при обработке сетевого пакета. Блокировка основного потока (Event Loop) более чем на 50 мс. Обнаружение захардкоженных данных для входа в коде.

7 Фаззинг-тестирование и безопасность

Обязательный этап фаззинг-тестирования на базе **AFL++** (American Fuzzy Lop) или **libFuzzer** по следующим причинам:

В методе `Server::onReadyRead()` происходит прямое чтение сырых байт из сокета в `QJsonDocument`. Злоумышленник может отправить:

- 10 гигабайт открывающих скобок `[[[. . .`
- Некорректную кодировку UTF-8.
- Пакет с обрезкой посередине байта.

Нарушив работу сервера, совершив атаку типа «Отказа в обслуживании» или получив удаленное выполнение кода (**RCE**)

7.1 Задачи разработчика по фаззингу

1. Выделить логику парсинга входящего сообщения в отдельную функцию, не привязанную к `QTcpSocket` (например, `processPayload(const QByteArray& data)`).
2. Написать Fuzz-таргет: точку входа `LLVMFuzzerTestOneInput`, которая принимает массив случайных байт от фаззера и скармливает его функции `processPayload`.
3. Скомпилировать проект с санитайзерами: `-fsanitize=address,undefined,fuzz`.
4. Запустить фаззер минимум на 24 часа машинного времени перед релизом. Любая выявленная ошибка должна быть оформлена как Critical Bug, исправлен и покрыт регрессионным тестом.

8 Требования к тестовой среде

- **Окружение:** Изолированный Docker-контейнер, дублирующий production-окружение (Ubuntu 22.04, Qt 6.x, PostgreSQL 15).

9 Обязанности

- **Разработчики:** Пишут код, пишут Unit-тесты (Qt Test), интегрируют санитайзеры и фаззинг-таргеты. Исправляют ошибки.
- **QA Инженер:** Проверяет архитектуру, проводит код-ревью тестов, пишет интеграционные Python-скрипты для проверки полного цикла работы (Регистрация -> Авторизация -> Расчеты). Настраивает пайплайны нагрузочного тестирования.

10 Расписание и Риски

Первым этапом внедряется модульное тестирование для `calc.cpp` и заглушки для `AuthManager`. До исправления архитектурной уязвимости с блокировкой `QEventLoop` при отправке писем через HTTP-клиент, интеграционное тестирование откладывается. **Риск:** Если блокировку не убрать (не перевести в асинхронный `QNetworkAccessManager`), сервер ляжет при нагрузке в 5 пользователей.